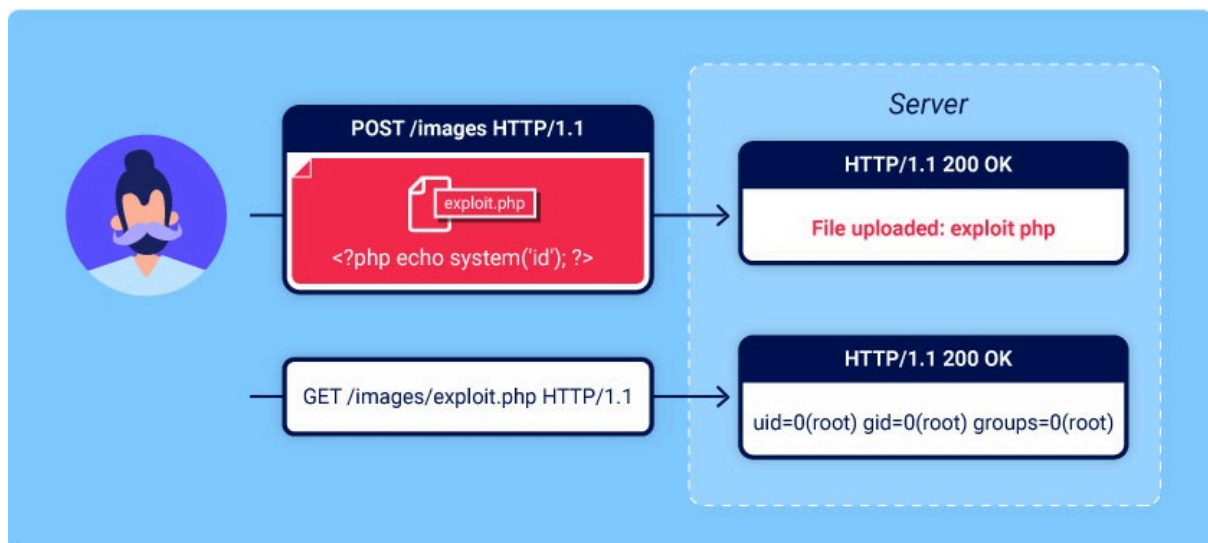


File Upload Vulnerabilities

I. Khái niệm và Tác động:

Lỗi hỏng này xảy ra khi máy chủ web cho phép người dùng tải lên tệp mà không kiểm tra đúng cách các thông tin như loại tệp, kích thước và nội dung. Điều này có thể cho phép kẻ tấn công tải lên tệp thực thi mã độc như web shell, cho phép họ kiểm soát máy chủ.



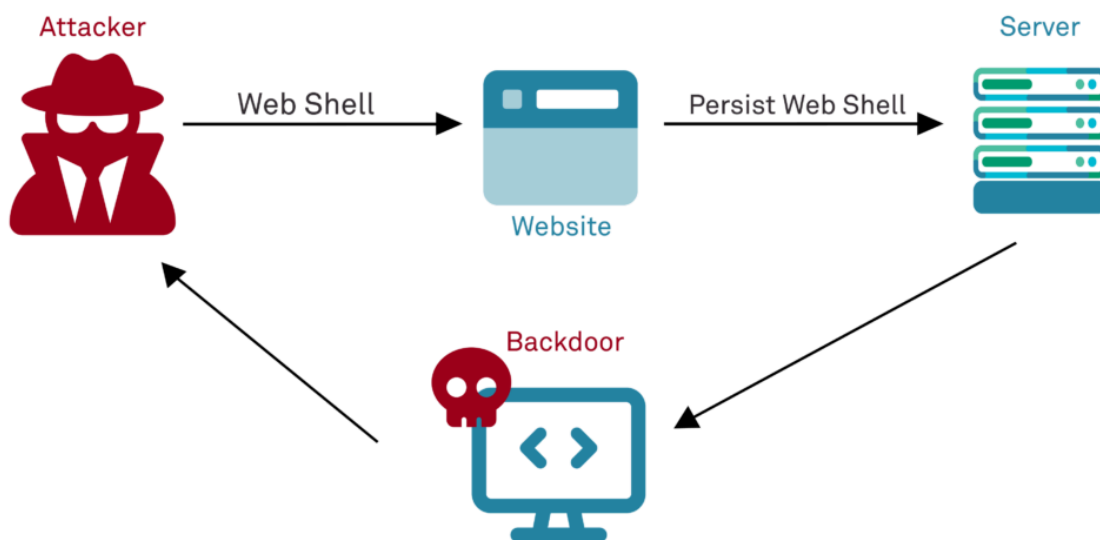
II. Khai thác xác thực tệp không đầy đủ

- **Cách khai thác:** Kẻ tấn công tải lên tệp thực thi bằng cách giả mạo Content-Type.
- **Ví dụ:** Tải lên tệp exploit.php với tiêu đề Content-Type: image/jpeg.
- **Các bước tấn công:**
 1. Tạo tệp PHP với mã độc: `<?php echo system($_GET['command']); ?>`
 2. Sử dụng công cụ như Burp Suite để thay đổi tiêu đề Content-Type thành image/jpeg.
 3. Tải lên tệp đến ứng dụng.
 4. Gửi yêu cầu GET để thực thi mã: `GET /uploads/exploit.php?command=id`

III. Thao tác cấu hình máy chủ



- **Cách khai thác:** Tạo tệp cấu hình (.htaccess hoặc web.config) để thay đổi cách máy chủ xử lý các tệp.
- **Ví dụ:** Tải lên tệp .htaccess để cho phép thực thi tệp .jpg như một tệp PHP.
- **Các bước tấn công:**
 1. Tạo tệp .htaccess với nội dung: AddType application/x-httpd-php .jpg
 2. Tải lên tệp .htaccess vào thư mục nơi các tệp người dùng được tải lên.
 3. Tải lên tệp exploit.jpg chứa mã PHP.
 4. Gửi yêu cầu để thực thi mã trong exploit.jpg.



IV. Sử dụng đuôi tệp giả mạo

- **Cách khai thác:** Thêm đuôi tệp hoặc ký tự ẩn để vượt qua xác thực.
- **Ví dụ:** Tải lên tệp exploit.php.jpg.
- **Các bước tấn công:**
 1. Tạo tệp PHP chứa mã độc.
 2. Đổi tên tệp thành exploit.php.jpg và tải lên.
 3. Gửi yêu cầu tới URL tệp và kiểm tra xem mã PHP có được thực thi không.



V. Tấn công dựa vào URL

- **Cách khai thác:** Tải lên tệp từ một URL không xác thực.
- **Ví dụ:** Sử dụng PUT để tải lên tệp từ URL.
- **Các bước tấn công:**
 1. Tạo tệp PHP với mã độc. Gửi yêu cầu PUT để tải lên tệp:
PUT /uploads/exploit.php HTTP/1.1

Host: vulnerable-website.com

Content-Type: application/x-httpd-php

Content-Length: 49

<?php echo file_get_contents('/path/to/file'); ?>
 2. Sau khi tải lên, gửi yêu cầu để thực thi mã.

VI. Khai thác qua XSS

- **Cách khai thác:** Tải lên tệp HTML hoặc SVG chứa mã JavaScript.
 - **Ví dụ:** Tải lên tệp payload.html chứa:
<script>alert('XSS Attack!');</script>
- **Các bước tấn công:**
 1. Tạo tệp payload.html chứa mã JavaScript.
 2. Tải lên tệp này vào ứng dụng web.
 3. Truy cập URL của tệp đã tải lên để kích hoạt mã JavaScript trong trình duyệt của người dùng khác.

VII. Khai thác thông qua lỗi phân tích tệp

- **Cách khai thác:** Lợi dụng lỗ hổng trong quá trình phân tích các tệp tải lên.
- **Ví dụ:** Tải lên tệp DOC hoặc XLS có chứa mã độc (XXE).
- **Các bước tấn công:**
 1. Tạo tệp DOC chứa mã độc trong phần XML.
 2. Tải lên tệp này qua chức năng tải lên của ứng dụng.



3. Kiểm tra xem có xảy ra lỗi phân tích nào không và khai thác nó để rò rỉ dữ liệu.

VIII. **Đảm bảo an toàn:**

Để ngăn chặn các lỗ hổng này, các biện pháp bảo mật như:

- Chỉ cho phép các đuôi tệp an toàn thông qua danh sách cho phép.
- Đảm bảo rằng tên tệp không chứa các ký tự có thể dẫn đến việc truy cập thư mục trái phép.
- Đổi tên tệp tải lên để tránh xung đột với các tệp khác.
- Không tải tệp lên hệ thống chính cho đến khi được xác thực đầy đủ.
- Sử dụng các framework bảo mật đã được kiểm chứng thay vì tự xây dựng phương pháp xác thực.

IX. **Tham khảo:**

<https://portswigger.net/web-security/learning-paths/file-upload-vulnerabilities>

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team